

Rapport PIM

Prune Mamalet

Garance Werner-Guyader

December-Janvier 2024

Plan du rapport :

- 1 Objectif du projet
- 2 Architecture de l'application en modules
- 3 Présentation des choix réalisés
- 4 Présentation des algorithmes et types de données
- 5 Démarche de test
- 6 Difficultés rencontrée et solutions
- 7 Organisation de l'équipe
- 8 Bilan technique de l'avancement
- 9 Bilan personnel et individuel

1 Objectif du projet

L'objectif de notre programme est de compresser un fichier texte à l'aide du codage d'Huffman. Il doit pouvoir afficher l'arbre et la table de codage d'Huffman qui permettent d'obtenir les codes correspondants aux symboles de notre texte.

2 Architecture de l'application en modules

- module_compresser : spécification et corps et des fonctions nécessaire à la compression.
- test_module_compresser : module de tests des fonctions et procédures de module—compresser.
- lca : module qui gère la définition et modification des dictionnaires (LCA).
- compresser : programme qui compresses le fichier texte à l'aide des modules précédents.

3 Présentation des choix réalisés

4 Présentation des algorithmes et types de données

4.1 Types de données

```
type T_Arbre is access T_Cellule1;  
  
type T_Cellule1 is record  
    Symbole : Character;  
    Frequence : Integer;  
    Code : T_Octet_Str;  
    Fils_Gauche : T_Arbre;  
    Fils_Droit : T_Arbre;  
end record;
```

Figure 1: type T_Arbre

Le Type Arbre permet de créer l'arbre d'Huffman en enregistrant le symbole et la fréquence et le code associés. Le type arbre est un pointeur vers un enregistrement (T_Cellule1) contenant les données souhaitées et les fils gauche et droit eux même de type T_Arbre.

```
type T_ListeChaine is access T_Cellule2 ;  
  
type T_Cellule2 is record  
    Arbre : T_Arbre;  
    Suivant : T_ListeChaine;  
end record;
```

Figure 2: type T_ListeChaine

Le type ListeChaine permet d'enregistrer différents arbres provisoires et donc de former l'arbre d'Huffman par regroupement. C'est également un pointeur vers un enregistrement (T_Cellule2).

4.2 Algorithmes

Il faut que l'on reprenne les raffinages qui ne sont plus adaptés à notre code.

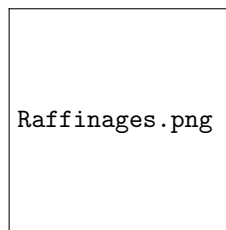


Figure 3: Raffinages

5 Démarche de test

Pour chacune de nos fonctions de module_compresser, on a créé des fonctions de test à l'aide la commande pragma assert. En plus, on a demandé d'afficher régulièrement l'arbre pour vérifier cette fonction (Afficher_Arbre). Lorsque tous les tests sont validés, le programme doit afficher "test ok".
Exemples :

```
procedure Tester_Suivant is
  Liste : T_ListeChaine;
begin
  Ajouter_Liste(Liste, Arbre1);
  Ajouter_Liste(Liste, Arbre2);
  pragma Assert (LArbre(Suivant(Liste))=Arbre2);
end Tester_Suivant;

procedure Tester_FinArbre is
begin
  pragma Assert(Fin_Arbre(Arbre1)=False);
  pragma Assert(Fin_Arbre(Arbre2)=True);
end Tester_FinArbre;
```

Figure 4: Tester suivant et fin de liste

```
procedure Tester_Fils is
  Fils_Droit : T_Arbre;
  Fils_Gauche : T_Arbre;
begin
  Initialiser_Arbre(Fils_Droit);
  Initialiser_Arbre(Fils_Gauche);
  Enregistrer_Arbre(Arbre2, p_1.To_Unbounded_String("1001010"), Fils_Gauche, Fils_Droit);
  Enregistrer_Arbre(Arbre3, 'a', 5, To_Unbounded_String("101101"), Fils_Gauche, Fils_Droit);
  Enregistrer_Arbre(Arbre2, p_2.To_Unbounded_String("1010"), Arbre3, Arbre2);
  pragma Assert(Arbre_Vide(Fils_Gauche(Arbre1)));
  pragma Assert(Fils_Droit(Arbre1)=Arbre2);
  Recuperer_Fils_Gauche(Fils_Gauche, Arbre1);
  Recuperer_Fils_Droit(Fils_Droit, Arbre1);
  pragma Assert(Arbre_Vide(Fils_Gauche));
  pragma Assert(Fils_Droit = Arbre2);
  Afficher_Arbre(Arbre1);
end Tester_Fils;
```

Figure 5: Tester fils

6 Difficultés rencontrée et solutions

Nous avons eu des difficultés pour trouver le bon type de données à utiliser pour pouvoir bien construire représenter l'arbre d'Huffman.

On a également eu du mal à comprendre ce qu'il fallait renvoyer dans le fichier compresser (des octets ou des codes en binaires ou des symboles etc).

Enfin la fonction Afficher_Arbre qui affiche l'arbre d'Huffman a été difficile à conceptualiser et coder.

9 Bilan personnel et individuel

Prune : J'ai du passer une après midi sur la conception des raffinages de *compresser*. J'ai passé 28 heures sur l'implémentation du code (dont 6 heures de projet en classe). Pour le rapport nous y avons pour l'instant passé 3 heures.

Garance : Peu de temps passé sur la conception des raffinages de *compresser* (je m'occupais de *décompresser*). Par la suite, environ 12h passé sur l'implémentation du code (6h de projet en classe et 6h pour le moment sur la fonction Afficher et d'autres correction de code).Quant au rapport, on a mis 3h pour l'instant.